



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 04

NOMBRE COMPLETO: Lechuga Castillo Sharenny Ixchel

N° de Cuenta: 319004252

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 04

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 13-09-2025

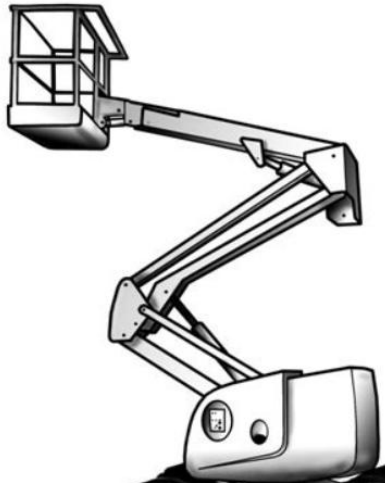
CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

Actividades realizadas:

Terminar de Construir la grúa con:

-brazo de 3 partes, 4 articulaciones, 1 canasta.



El código que obtenemos es el siguiente:

```
354 //CREANDO LA CABINA DE LA GRÚA
355 //para reiniciar la matriz de modelo con valor de la matriz identidad
356 model = glm::mat4(1.0f);
357 model = glm::translate(model, glm::vec3(0.0f, 4.0f, -4.0f));
358 modelaux = model;
359 model = glm::scale(model, glm::vec3(4.0f, 2.0f, 2.0f));
360 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
361 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
362 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
363 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
364 color = glm::vec3(1.0f, 0.0f, 0.0f); // rojo
365 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); // para cambiar el color del objetos
366 meshList[0]->RenderMesh();
367 model = modelaux;
368
369 //ARTICULACION CABINA-BRAZO 1
370 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
371 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
372
373 sp.render();
374 shaderList[0].useShader();
375 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // seguro
376 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
377
378 //-----Creando el brazo de una grúa-----
379 //articulacion1 hasta articulacion5 sólo son puntos de rotación o articulación, en este caso no di
380
381 //primer brazo que conecta con la cabina
382 model = glm::rotate(model, glm::radians(135.0f), glm::vec3(0.0f, 0.0f, 1.0f));
383 model = glm::translate(model, glm::vec3(2.5f, 0.0f, 0.0f));
384 modelaux = model;
385 model = glm::scale(model, glm::vec3(5.0f, 1.0f, 1.0f));
386
387 //rotación alrededor de la articulación que une con la cabina
388 //Traslación inicial para posicionar en -2 a los objetos
389 //model = glm::translate(model, glm::vec3(0.0f, 0.0f, -4.0f));
390 //otras transformaciones para el objeto
391 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
```

```

392 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
393 //se programe cambio entre proyección ortogonal y perspectiva
394 color = glm::vec3(1.0f, 1.0f, 0.0f);
395 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
396 meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
397 //meshList[3]->RenderMeshGeometry(); //dibuja las figuras geométricas cilindro, cono, pirámide base cuad
398 //sp.render(); //dibuja esfera
399 model = modelaux;
400 //SEGUNDA ARTICULACIÓN
401 model = glm::translate(model, glm::vec3(2.5f, 0.0f, 0.0f));
402 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));
403 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
404 sp.render();
405
406 //-----segundo brazo-----
407
408 //Comentar y se modifica para agregar la jerarquía:
409 //usar una matriz temporal o auxiliar
410
411 //model = glm::mat4(1.0);
412 model = glm::translate(model, glm::vec3(0.0f, -2.5f, 0.0f));
413
414 //Traslación inicial para posicionar en -Z a los objetos
415 //model = glm::translate(model, glm::vec3(0.0f, 0.0f, -4.0f));
416 //otras transformaciones para el objeto
417 modelaux = model;
418 model = glm::scale(model, glm::vec3(1.0f, 5.0f, 1.0f));
419 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
420 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
421 //se programe cambio entre proyección ortogonal y perspectiva
422 color = glm::vec3(1.0f, 1.0f, 0.0f);
423 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
424 meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular

```

```

426 //ARTICULACION3
427 //SEGUNDA ARTICULACIÓN
428 model = modelaux;
429 model = glm::translate(model, glm::vec3(0.0f, -2.5f, 0.0f));
430 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, 1.0f));
431 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
432 sp.render();
433
434 //-----Tercer brazo-----
435 model = glm::translate(model, glm::vec3(0.0f, -3.2f, 0.0f));
436
437 //Traslación inicial para posicionar en -Z a los objetos
438 //model = glm::translate(model, glm::vec3(0.0f, 0.0f, -4.0f));
439 //otras transformaciones para el objeto
440 modelaux = model;
441 model = glm::scale(model, glm::vec3(1.0f, 5.0f, 1.0f));
442 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
443 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
444 //se programe cambio entre proyección ortogonal y perspectiva
445 color = glm::vec3(1.0f, 1.0f, 0.0f);
446 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
447 meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
448 //-----ARTICULACION 4-----
449 model = modelaux;
450 model = glm::translate(model, glm::vec3(0.0f, -3.2f, 0.0f));
451 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, 1.0f));
452 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
453 sp.render();

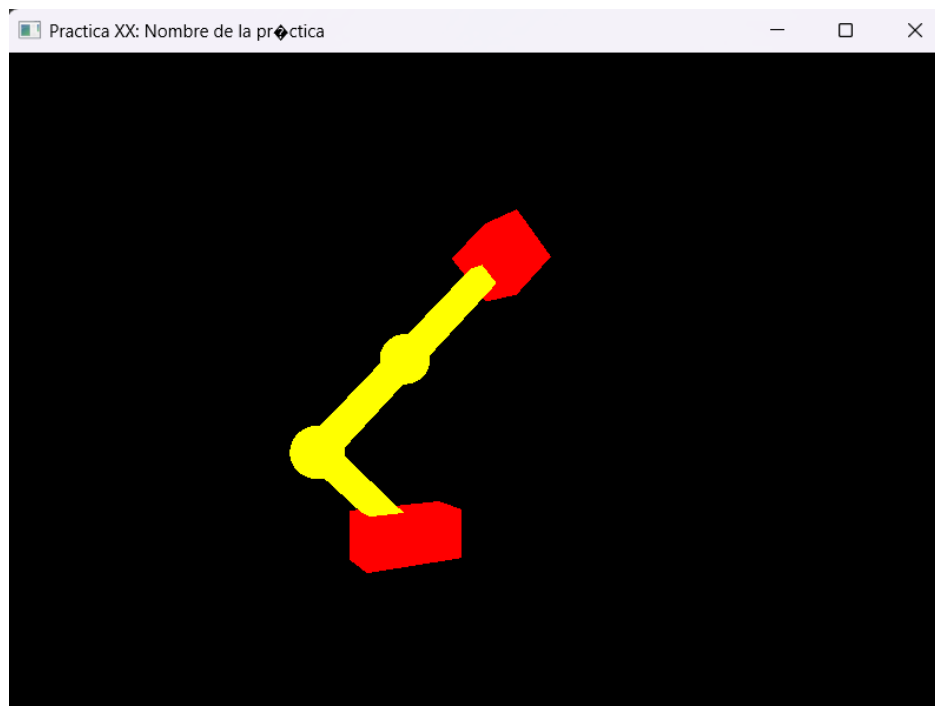
```

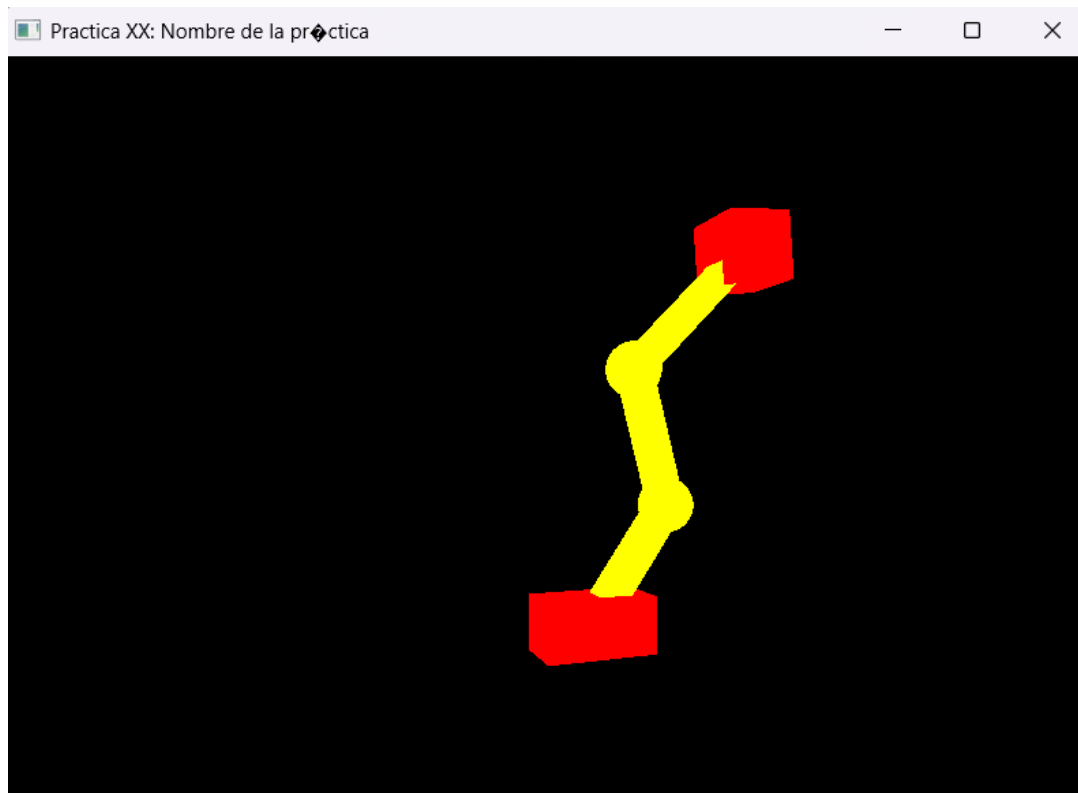
```

455 //-----Canasta-----
456 model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
457 modelaux = model;
458 model = glm::scale(model, glm::vec3(2.5f, 2.5f, 2.5f));
459 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
460 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
461 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
462 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
463 color = glm::vec3(1.0f, 0.0f, 0.0f); // rojo
464 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); // para cambiar el color del objetos
465 meshList[0] -> RenderMesh();
466 model = modelaux;
467
468 shaderList[0].useShader();
469 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // seguro
470 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
471
472 glUseProgram(0);
473 mainWindow.swapBuffers();
474 }
475 return 0;
476 }

```

El ejecutable que obtenemos es el siguiente:





Problemas presentados:

Tuve problemas al entender donde se encontrabas las articulaciones, al no ir se me complico bastante como estar buscando y entendiendo el código. Ya no hay muchos problemas en las coordenadas porque gracias a la practica ya las puedo instalar de manera correcta y mucho más rápido que al principio o que el semestre pasado.

Conclusión:

- a. Los ejercicios de la clase: No fue complejo, creo que ha sido el menos complejo que hemos realizado, o no sé si yo entendí mal lo que tenía que entregar, pero se me hizo super sencillo realizarlo.
- b. Comentarios generales: En esta ocasión no puedo dar comentarios sobre eso, ya que no pude asistir pero supongo que han sido como siempre, la explicación es rápida y trata de hacerla lo más clara posible pero ahora si ya empiezan las prácticas más difíciles.